

Topic 2: Variables, Data Types, and Input/Output

What is a Variable?

A variable is a named location in memory that stores a value. When you declare `int age = 20;`, the computer reserves a small block of RAM, names it "age", and stores the number 20 there. Every variable has a name, a data type, and a value.

Declaring and Initializing Variables

```
```cpp
int score = 100; // integer
double price = 9.99; // decimal number
char grade = 'A'; // single character
bool isActive = true; // true or false
```
```

You can declare without initializing (`int x;`) but uninitialized variables contain garbage values – always initialize.

Fundamental Data Types

| Type | Size | Range / Use |
|-----------|---------|--|
| int | 4 bytes | Whole numbers: -2,147,483,648 to 2,147,483,647 |
| float | 4 bytes | Decimal numbers, ~7 digits of precision |
| double | 8 bytes | Decimal numbers, ~15 digits of precision (preferred) |
| char | 1 byte | Single character stored as ASCII value |
| bool | 1 byte | true (1) or false (0) |
| long long | 8 bytes | Very large whole numbers |

Use the `sizeof` operator to check the size of any type: `cout << sizeof(int);`

Constants

A constant is a variable whose value cannot change after it is set.

```
```cpp
const double PI = 3.14159;
const int MAX_STUDENTS = 50;
```
```

Use constants for values that should never change. This prevents accidental modification and makes code easier to read.

Naming Rules and Best Practices

- Must start with a letter or underscore, not a number
- Cannot use reserved keywords (int, for, while, etc.)
- Case-sensitive: `age` and `Age` are different variables
- Use camelCase for variables: `studentName`, `totalScore`
- Use ALL_CAPS for constants: `MAX\_SIZE`
- Use descriptive names – `speed` is better than `s`

Input and Output

```
```cpp
int age;
cout << "Enter your age: "; // output to screen
cin >> age; // read input from keyboard
cout << "You are " << age << " years old." << endl;
```
```

- `cout <<` outputs to screen
- `cin >>` reads from keyboard
- You can chain multiple outputs: `cout << "Hello" << " " << "World";`

Escape Sequences

| Sequence | Meaning |
|----------|--------------|
| `\\n` | New line |
| `\\t` | Tab |
| `\\\\` | Backslash |
| `\\"` | Double quote |
| `\\`` | Single quote |

The string Type

```
```cpp
#include <string>
string name = "Ali";
cout << "Hello, " << name << endl;
cout << "Length: " << name.length() << endl;
```
```

Strings store text. They are more powerful than char arrays and much easier to use.

Type Casting

Converting one data type to another:

```
```cpp
int a = 5, b = 2;
double result = (double)a / b; // explicit cast → 2.5
// without cast: 5/2 = 2 (integer division loses the decimal)
```
```

Implicit conversion happens automatically when assigning a smaller type to a larger type (int → double). Explicit conversion uses `(type)` syntax.

```
---
```